

703088 (2026WS) - PS Distributed Systems

Distributed and Parallel Systems Group
University of Innsbruck
Instructors: Juan Aznar Poveda, Zahra Najafabadi Samani, and Marlon Etheredge
Author(s): Marlon Etheredge and Juan Aznar Poveda
In case of questions regarding this task, please contact the author(s) ({marlon.etheredge, juan.aznar-poveda}@uibk.ac.at).

EX03 - Network Topologies, Peer Discovery & Byzantine Agreement

Introduction

In this exercise, you will explore how processes connect and communicate in a distributed system, transitioning from rigid static addressing to dynamic peer discovery. Distributed systems rely on various network topologies (e.g., centralized, hierarchical, or mesh/P2P). While mesh and P2P architectures handle dynamic changes gracefully and eliminate single points of failure, they require nodes to discover each other automatically.

Systems such as Zenoh, libp2p, or Hazelcast already leverage built-in P2P discovery protocols to automatically form network topologies without explicitly hard-coding the IP address of every other process.

In open, decentralized systems, nodes may behave arbitrarily (Byzantine faults): they can lie, send conflicting messages, or collude. Reaching agreement in the presence of such faults is the Byzantine generals problem, and it is strictly harder than crash-fault tolerance.

In this exercise, use a programming language and a P2P framework of your choice (e.g., Zenoh). Implement the distributed system as separate programs that communicate over a network without hard-coding other application nodes' addresses.

Task 1 - Discovering peers

Consider three application nodes (A, B, and C) that need to discover each other dynamically to form a mesh.

- **1.1).** Implement a program that starts a P2P session (e.g., using your chosen framework's discovery mechanisms). Each instance should identify itself with a unique name and periodically display information about the available peers it has discovered. Run three instances of the program in separate terminals and verify that the nodes discover each other automatically.
- **1.2).** Run the following experiment to observe dynamic behavior:
  - Stop Node B and wait a few seconds.
  - Observe the output of Nodes A and C.
  - Start Node B again and observe the system.

  Record your observations and answer the following questions:
  - How does a node learn that another node is available?
  - What happens when Node B leaves, and what happens when it joins again?
  - Your discovery mechanism tells you a node is reachable. Can you trust that a discovered node is who it claims to be, or that its messages are truthful?

Task 2 - Exchange messages between peers

Extend your program so that the discovered nodes can exchange data.

- **2.1).** Implement a publish/subscribe mechanism where each node periodically publishes a message (e.g., "Hello from Node A") to a shared topic or key expression.

Ensure that all other nodes subscribe to this topic and print the messages they receive. Demonstrate that they can exchange messages without having explicitly configured addresses.

- **2.2).** Answer the following conceptual questions:
  - What is the main difference between static configuration and dynamic discovery?
  - Why is the publish/subscribe pattern useful for communication between distributed processes?
  - Is peer discovery sufficient to make an application fault-tolerant? Explain briefly.

Task 3 - The Byzantine Generals Problem

In the Byzantine generals problem (Lamport, Shostak & Pease, 1982), a commander sends an order ("attack" or "retreat") to two lieutenants. Up to one of the three generals may be a traitor and behave arbitrarily (e.g., send conflicting orders). The loyal generals must satisfy two properties:

- **Agreement:** all loyal lieutenants decide on the same order.
- **Validity:** if the commander is loyal, all loyal lieutenants decide on the commander's order.

With only three nodes, these properties cannot both be guaranteed when the traitor is the commander. In this task, you will implement the three-node scenario, observe the impossibility, and then extend it to four nodes, where agreement becomes possible. This is the smallest example of why decentralization without a trusted coordinator is fundamentally harder than it looks.

- **3.1).** Implement the three-node scenario as separate programs communicating over the network (reuse the publish/subscribe mechanism from Task 2). One node acts as the **commander** and sends an order to the two **lieutenants**. Each lieutenant then forwards the order it received to the other lieutenant (oral messages, one round of exchange). Each lieutenant decides on an order based on the values it has received. Select the role (commander/lieutenant) and whether the node is a traitor via a command-line argument (e.g., `--role commander --traitor`). A traitor may behave arbitrarily; for example, the commander sends "attack" to one lieutenant and "retreat" to the other, or a lieutenant forwards a value different from the one it received.
- **3.2).** Run two experiments with three nodes and record the decisions of both lieutenants:
  - **All loyal:** verify that both lieutenants decide on the commander's order.
  - **Traitor commander:** the commander sends "attack" to one lieutenant and "retreat" to the other. Observe that the loyal lieutenants can decide differently; agreement is impossible with three nodes. Explain why a lieutenant cannot tell the two situations apart: from its point of view, "the commander is a traitor" and "the other lieutenant is a traitor" look identical.
- **3.3).** Extend your implementation to **four nodes** (one commander, three lieutenants). Each lieutenant now exchanges its received order with both other lieutenants, and each lieutenant decides by **majority vote** over the three values it holds (the order received from the commander plus the two forwarded orders). Repeat both experiments from 3.2 (all loyal, and one traitor; try both a traitor commander and a traitor lieutenant). Verify that in all cases the loyal lieutenants agree on the same order.
- **3.4).** Answer the following conceptual questions:
  - Why does majority voting succeed with three lieutenants but fail with only two?
  - In your implementation the commander is a designated, central node. Tasks 1–2, by contrast, built a fully decentralized mesh without a coordinator. What does the commander provide that makes agreement possible with four nodes, and why does the problem become harder when all nodes are equal?
  - Your four-node implementation guarantees agreement, but can it identify *which* node is the traitor? Explain what a loyal node can and cannot observe.
  - By default, nodes often use automatic multicast scouting to discover each other on a local network. In restricted networks (or wide-area networks) where multicast is blocked. How could nodes communicate in such cases in practice? Investigate whether the middleware you selected offers such a feature.

Submission

Submit a PDF file with your solution and your code. You can optionally use the markdown template provided in OLAT (`Submission_Template.md`) and convert it to PDF.

Your report should address all tasks and questions specified in the exercise. Depending on the exercise, this may include:

- theoretical questions and explanations;
- a description of your implementation;
- source code snippet;
- experimental results, observations, or screenshots;
- answers to the discussion questions;
- for Task 3: the decisions of the loyal lieutenants in each experiment (all loyal vs. traitor, with three and with four nodes).

Focus your report on the **distributed-systems concepts, your design decisions, and your observations**. Keep the report concise and avoid documenting routine installation steps unless they are relevant to the exercise.

The submission should follow the following structure and naming convention:

```
|-- exc0X_report_csXXXXXX.pdf
|-- exc0X_source_csXXXXXX.zip
  |-- README.md
  |-- sample_file_a.py
  |-- sample_file_b.py
  |-- sample_file_c.py
  |-- byzantine_node.py
```

Deadline

05.11.2026 12:00h

Presentation

06.11.2026